

Lab 1 — Requirements Engineering & UML Use-Case Modelling

Problem Statement #41 — OpenAPI Mock Server Generator

PES1UG24AM197

PES University — Dept. of CSE

August 2026

Requirements Specification

Project: OpenAPI Mock Server Generator **Problem Statement:** #41 — Developer Tools & IT Operations **Actors / Stakeholders:** API Developer, QA Engineer

A developer-productivity tool that ingests OpenAPI 3.0 (YAML/JSON) specifications, generates dynamic mock REST endpoints with schema validation, and simulates configurable response latency.

1. Functional Requirements

FR-001 — Ingest & Parse OpenAPI Specification

Field	Detail
ID	FR-001
Type	Functional
Priority	High
Description	The system shall allow the API Developer to import an OpenAPI 3.0 specification (YAML or JSON) via file upload or URL, parse it, and validate its structural conformance to the OpenAPI 3.0 standard, reporting any syntax or schema errors together with their location.
Acceptance Criteria	Pass: A valid OpenAPI 3.0 YAML/JSON file is parsed successfully and its paths, operations, and component schemas are extracted; a malformed spec is rejected with a descriptive error message identifying the fault location. Fail: A malformed spec is accepted silently, or a valid spec fails to parse.
Rationale	Reliable ingestion is the foundation of the tool. Without correct parsing and validation, every downstream mock endpoint would be inaccurate, so this must be trustworthy before anything else is built.

FR-002 — Generate Dynamic Mock Endpoints

Field	Detail
ID	FR-002
Type	Functional
Priority	High
Description	The system shall automatically generate live HTTP mock endpoints for every path and operation (GET, POST, PUT, PATCH, DELETE) defined in the parsed specification, returning randomized JSON payloads that conform to the declared response schema (correct field names, data types, required fields, and constraints such as <code>enum</code> and <code>format</code>).
Acceptance Criteria	Pass: Each mocked endpoint responds on the correct method + path with a JSON body matching the declared schema types; a request to an undefined path returns <code>404 Not Found</code> . Fail: An invalid/undefined endpoint path returns <code>200 OK</code> , or a returned payload violates the declared schema types.
Rationale	The core value of the tool is providing working, schema-accurate endpoints so front-end and integration work can proceed in parallel before the real API exists.

FR-003 — Validate Incoming Request Against Schema

Field	Detail
ID	FR-003
Type	Functional
Priority	High
Description	The system shall validate each incoming request (path parameters, query parameters, headers, and request body) against the operation's declared schema and return an appropriate <code>4xx</code> response (e.g., <code>400 Bad Request</code>) with a validation error report when the request does not conform.
Acceptance Criteria	Pass: A request that violates the schema (missing required field, wrong data type) yields a <code>400</code> with error details; a conforming request yields the mocked success response. Fail: A schema-violating request returns a <code>2xx</code> success response.
Rationale	Request validation lets the QA Engineer confirm that their client honours the API contract and surfaces integration defects early, which is a primary reason for using a mock server.

FR-004 — Configure Simulated Response Latency

Field	Detail
ID	FR-004
Type	Functional
Priority	Medium
Description	The system shall allow the user to configure simulated response latency — globally or per endpoint — as either a fixed delay or a randomized range (min–max ms), so that real-world or degraded network conditions can be emulated.
Acceptance Criteria	Pass: With latency set to <i>N</i> ms, the measured response time equals <i>N</i> ms within tolerance; a range configuration produces delays within <code>[min, max]</code> . Fail: Configured latency has no measurable effect on response timing.
Rationale	Testing client timeout handling, loading states, and resilience requires controllable, repeatable latency that a live API cannot reliably provide.

FR-005 — Configure Custom Responses & Status Codes

Field	Detail
ID	FR-005
Type	Functional
Priority	Medium
Description	The system shall allow the user to override the default generated response for a given endpoint by specifying a custom example payload and/or forcing a specific HTTP status code (e.g., <code>201</code> , <code>401</code> , <code>429</code> , <code>500</code>) to simulate defined success and error scenarios.
Acceptance Criteria	Pass: An endpoint configured to return a custom <code>500</code> payload consistently returns that status and body; removing the override restores the default generated behaviour. Fail: The configured override is ignored and the default response is returned instead.
Rationale	QA needs deterministic error and edge-case scenarios to exercise client error-handling paths that are difficult or unsafe to trigger against a live API.

2. Non-Functional Requirements

NFR-001 — Throughput & Latency Accuracy

Field	Detail
ID	NFR-001
Type	Performance
Priority	High
Description	The mock server engine shall sustain at least 1,000 mock API requests per second while honouring user-configured simulated latency accurate to within ± 10 ms , under simulated peak load.
Acceptance Criteria	Pass: Load/benchmark tests (e.g., k6 or JMeter) confirm ≥ 1,000 req/s sustained throughput with p95 latency deviation ≤ 10 ms from the configured value. Fail: Sustained throughput < 1,000 req/s, or latency deviation > 10 ms.
Rationale	Mock servers used in CI and performance pipelines must not become the bottleneck and must faithfully reproduce configured timing, otherwise test results are misleading.

NFR-002 — Reliability & Security

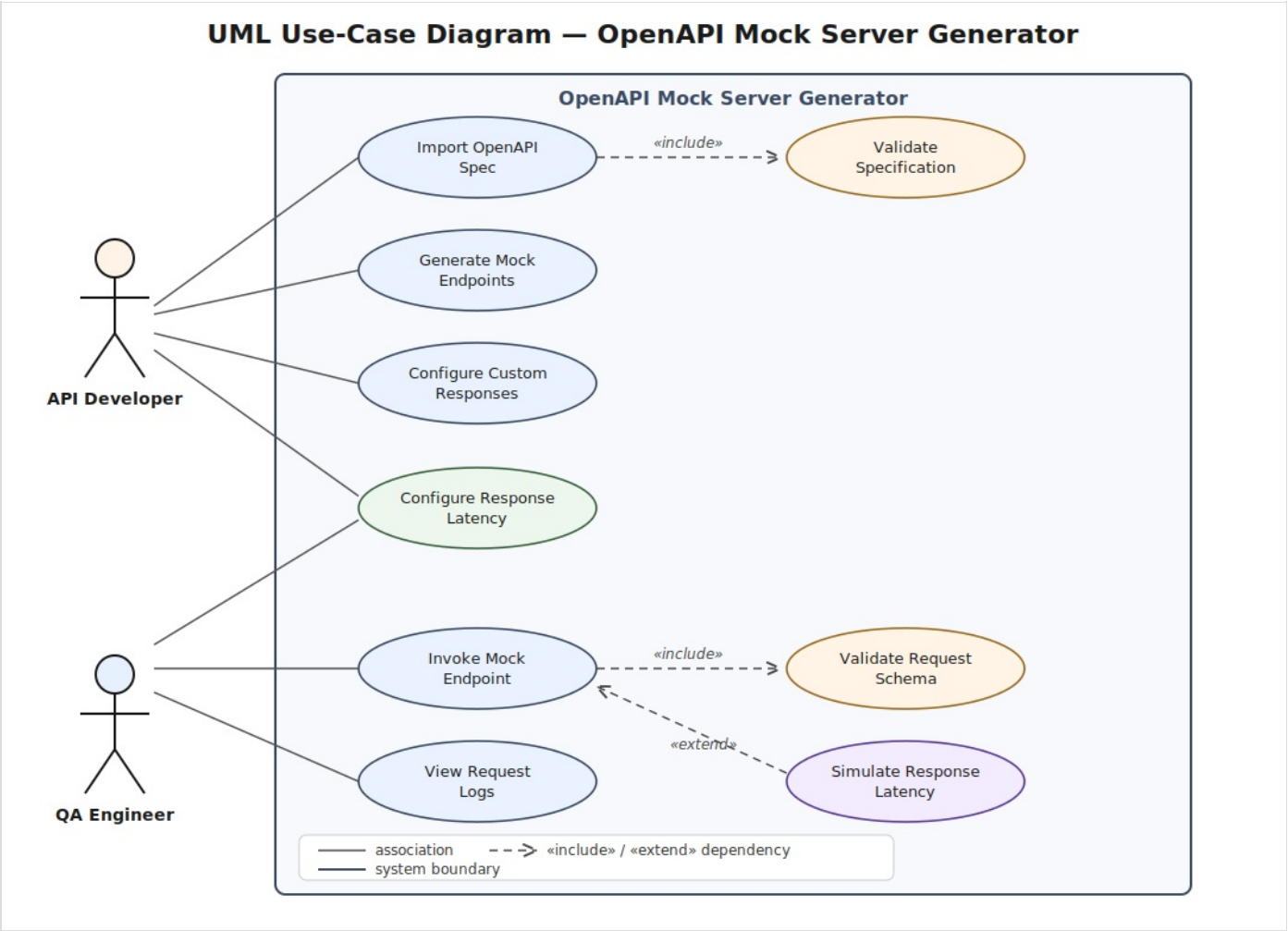
Field	Detail
ID	NFR-002
Type	Reliability & Security
Priority	High
Description	The system shall run each mock instance in isolation, reject malformed or oversized specification uploads (> 5 MB) gracefully without crashing, sanitize all inputs to prevent injection, expose no real backend data, and maintain ≥ 99.5% availability across a continuous 8-hour test session.
Acceptance Criteria	Pass: Fault injection (malformed/oversized specs, fuzzed requests) never crashes the server; measured availability ≥ 99.5% over 8 hours; a security scan reports no successful injection. Fail: Any malformed input crashes the process, availability < 99.5%, or an injection succeeds.
Rationale	A tool embedded in developer and QA workflows must be dependable and safe; crashes or vulnerabilities would erode trust and could leak or corrupt test data.

Requirements Traceability Summary

ID	Title	Type	Priority	Primary Actor
FR-001	Ingest & Parse OpenAPI Specification	Functional	High	API Developer
FR-002	Generate Dynamic Mock Endpoints	Functional	High	API Developer
FR-003	Validate Incoming Request Against Schema	Functional	High	QA Engineer
FR-004	Configure Simulated Response Latency	Functional	Medium	API Developer / QA Engineer
FR-005	Configure Custom Responses & Status Codes	Functional	Medium	API Developer / QA Engineer
NFR-001	Throughput & Latency Accuracy	Performance	High	—
NFR-002	Reliability & Security	Reliability & Security	High	—

UML Use-Case Diagram

Project: OpenAPI Mock Server Generator



Actors

Actor	Role
API Developer	Imports specifications, generates the mock server, and configures endpoint behaviour.
QA Engineer	Exercises the generated mock endpoints during testing and inspects request logs.

Use Cases

Use Case	Actor(s)	Related Requirement
Import OpenAPI Spec	API Developer	FR-001
Generate Mock Endpoints	API Developer	FR-002
Configure Custom Responses	API Developer	FR-005
Configure Response Latency	API Developer, QA Engineer	FR-004
Invoke Mock Endpoint	QA Engineer	FR-002 / FR-003
View Request Logs	QA Engineer	FR-003
Validate Specification	(included)	FR-001
Validate Request Schema	(included)	FR-003
Simulate Response Latency	(extension)	FR-004

Relationships

- «include»** — *Import OpenAPI Spec* **includes** *Validate Specification*. Validation is an unconditional, mandatory sub-step of every import, so it is factored out as an included use case.
- «include»** — *Invoke Mock Endpoint* **includes** *Validate Request Schema*. Every incoming request is validated against the schema, so this behaviour is always executed as part of invocation.

- **«extend»** — *Simulate Response Latency* **extends** *Invoke Mock Endpoint*. Latency simulation is applied **conditionally**, only when the user has configured a delay for that endpoint; the base use case is complete without it, which is exactly what an extension models.

PlantUML Source (regenerable)

```
@startuml
left to right direction
skinparam actorStyle awesome
skinparam packageStyle rectangle

actor "API Developer" as dev
actor "QA Engineer" as qa

rectangle "OpenAPI Mock Server Generator" {
    usecase "Import OpenAPI Spec" as UC1
    usecase "Generate Mock Endpoints" as UC2
    usecase "Configure Custom Responses" as UC3
    usecase "Configure Response Latency" as UC4
    usecase "Invoke Mock Endpoint" as UC6
    usecase "View Request Logs" as UC7
    usecase "Validate Specification" as UC5
    usecase "Validate Request Schema" as UC8
    usecase "Simulate Response Latency" as UC9
}

dev --> UC1
dev --> UC2
dev --> UC3
dev --> UC4
qa --> UC6
qa --> UC7
qa --> UC4

UC1 ..> UC5 : <<include>>
UC6 ..> UC8 : <<include>>
UC9 ..> UC6 : <<extend>>
@enduml
```

Use-Case Flow Specification

Use Case: UC-06 — Invoke Mock Endpoint **Primary Actor:** QA Engineer **Related Requirements:** FR-002, FR-003, FR-004 **Included:** Validate Request Schema **Extended by:** Simulate Response Latency

Brief Description

The QA Engineer sends an HTTP request to a generated mock endpoint. The system matches the request to a route defined in the OpenAPI specification, validates the request against the operation schema, produces a schema-compliant JSON response, optionally applies a configured latency, returns the response, and records the exchange in the request log.

Preconditions

- A valid OpenAPI 3.0 specification has been imported and parsed (FR-001).
- The mock server is running and its endpoints have been generated and are live (FR-002).
- The QA Engineer knows the base URL and has network access to the mock server.

Postconditions

On success: 1. An HTTP response with a schema-compliant body and the appropriate status code is returned to the caller. 2. The request and response metadata (method, path, status, timestamp, latency) are recorded in the request log.

On failure: 1. An appropriate `4xx` (validation) or `404` (no matching route) response is returned. 2. The failed request and its rejection reason are recorded in the request log.

Main Success Scenario

- The QA Engineer sends an HTTP request (method + path + optional params/body) to a mock endpoint.
- The system matches the request to a route and operation defined in the specification.
- The system **includes** → **Validate Request Schema**: it checks path/query parameters, headers, and the request body against the operation's declared schema, and confirms the request is valid.
- The system generates a randomized JSON payload that conforms to the operation's declared response schema, and selects the appropriate success status code (e.g., `200` / `201`).
- «extend»** → **Simulate Response Latency**: if a latency value is configured for this endpoint, the system waits for the configured delay before responding.
- The system returns the HTTP response to the QA Engineer.
- The system records the request/response exchange in the request log.

Alternate Flow

A1 — Request Fails Schema Validation

Triggered at step 3 when the request does not conform to the schema (e.g., a required field is missing or a field has the wrong data type).

- The system halts response generation for the invalid request.
- The system builds a `400 Bad Request` response containing a validation error report that identifies the offending field(s) and the reason for rejection.
- The system returns the `400` response to the QA Engineer.
- The system records the rejected request and its validation error in the request log.
- The use case ends. (The QA Engineer may correct the request and retry, returning to step 1 of the Main Success Scenario.)